

03 · Manual técnico

Para quién: mantenimiento y DevOps. Qué cuenta: cómo levantar el proyecto en una máquina, cómo corre la ingesta, cómo se despliega y qué hacer cuando algo falla.

Verificado el 5 de octubre de 2026 · commit `2e66668`.

Antes de cambiar cualquier cosa en `pulso/`, lee `AGENTS.md` completo. Varias «mejoras» obvias —rellenar un hueco con cero, añadir una marca de hora, cachear texto de comentarios— rompen una obligación legal o la promesa central del producto.

1. Entorno local

Requisitos

Herramienta	Versión
Python	3.11 (el servicio de tono usa 3.12)
Node	22
pnpm	11.18
Git	con <code>.gitattributes</code> respetado (fuerza LF)

En PowerShell, antes de cualquier comando de Python:

```
$env:PYTHONUTF8 = "1"
```

El pipeline

```
python -m pip install -r requirements.txt
```

`requirements.txt` es **una línea** (Scrapy). Todo lo demás —RSS, HTTP, el validador, los temas— es biblioteca estándar. No se agregan `requests`, `pandas`, `feedparser` ni `beautifulsoup4`.

El modelo de tono es opcional y pesa (~430 MB más torch en CPU):

```
python -m pip install torch --index-url https://download.pytorch.org/whl/cpu
```

```
python -m pip install -r requirements-modelo.txt
```

Sin él todo corre con `--metodo ninguno` y las notas salen con `postura: null`.

El tablero

```
pnpm --dir web install
```

```
pnpm --dir web dev
```

Abre `http://localhost:3000`. `dev` copia primero `data/` a `web/public/data/`; **solo al arrancar**. Si el pipeline corrió después, o hiciste `git pull`, vuelve a copiar:

```
pnpm --dir web datos
```

Variables locales: copia `web/.env.example` a `web/.env.local`. Para entrar sin Entra ID en desarrollo, `ACCESO_SIN_ENTRA=true` (solo funciona fuera de producción). El pipeline lee también `.env`, `.env.local`, `web/.env` y `web/.env.local`, en ese orden; el entorno real gana.

Cuidado: si tu `.env` tiene `APIFY_TOKEN`, un sondeo «de prueba» es una llamada real y pagada.

La comprobación por omisión

```
python -m pulso correr --sin-red --salida "$TEMP/pulso-prueba"
```

Corre todo el pipeline sin red sobre 15 titulares reales. **Pasa siempre** `--salida`: sin ella reescribe `data/` con esos 15 titulares.

2. Los comandos

`python -m pulso <verbo>`. Los que cuestan dinero están marcados.

Verbo	Escribe	Cron	Costo
correr	notas.json, fuentes.json, estado.json, temas.json, archivo/	Sí	Gratis
comunicados	comunicados.json	Sí	Gratis
youtube	youtube.json	Sí	Gratis
indicadores	indicadores.json (se salta si tiene < 7 días)	Sí	Gratis

Verbo	Escribe	Cron	Costo
<code>gasto-electoral</code>	<code>gasto-electoral.json</code> , <code>financiamiento-partidos.json</code>	Lunes	Gratis
<code>validar</code>	Nada; sale con 1 si hay error	Sí, tres veces	Gratis
<code>apify --verificar</code>	Nada; revisa token y catálogo	Sí	Gratis
<code>redes</code>	<code>redes.json</code> + comentarios	Sí	Apify
<code>tiktok</code>	<code>tiktok.json</code> + comentarios	Sí	Apify
<code>facebook</code>	<code>facebook.json</code> + comentarios	Sí	Apify
<code>tendencias</code>	<code>tendencias.json</code>	Sí	Apify
<code>conversacion</code>	<code>conversacion.json</code>	Solo con <code>YOUTUBE_HABILITADO</code>	Cuota de la API
<code>consultas</code>	<code>consultas.json</code> + comentarios	No, a mano	Apify (gratis con <code>--sin-cosecha</code>)
<code>expediente-redes</code>	<code>web/src/lib/expedientes/<id>-redes.json</code> + comentarios	No, a mano	Apify ; <code>--estimar</code> primero
<code>publicidad-meta</code>	<code>pauta-meta.json</code>	No, a mano	Gratis
<code>delegaciones --actualizar</code>	<code>config/delegaciones-tijuana.json</code> y el mapa TS	No	Gratis
<code>tono --servir</code>	Nada; servidor local de tono en :8765	No	Gratis
<code>sitio, servir, evaluar</code>	El tablero anterior y la línea base	No	Gratis

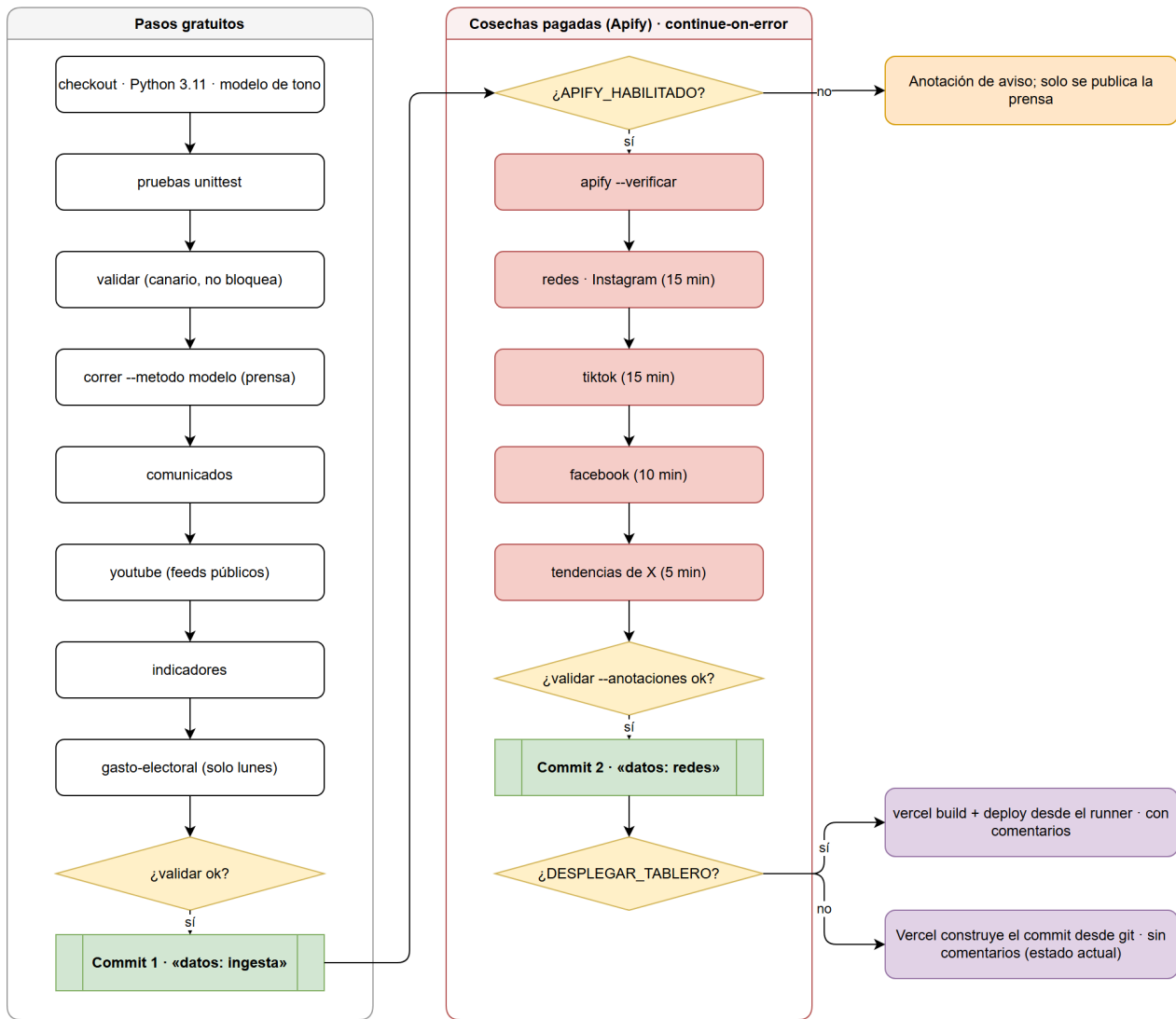
Los sondeos (`--sondear`, `--probar`, `--muestra`) **no escriben nada** y son el paso obligatorio antes de activar una cuenta (ver manual del administrador).

Códigos de salida. `correr` devuelve **2** solo cuando ninguna fuente respondió (apagón total); una falla parcial es normal y sale 0. `validar` devuelve 1 ante cualquier error. No se suavizan.

3. El ciclo de la ingesta

`.github/workflows/pulso.yml`, cada seis horas al minuto 17 (`17 */6 * * *`), más los lunes a las 18:41 UTC para el gasto electoral. También corre con cada push a `main` que no sea solo Markdown o `docs/`, y a mano desde la pestaña Actions.

Pulso N33 · Ciclo de la ingesta (pulso.yml, cada 6 h al minuto 17)



Límite del trabajo: 75 min, por encima de la suma de los límites de cada paso pagado, para que siempre se guarde lo ya cosechado.

Editable en draw.io: [diagramas/ciclo-de-ingesta.drawio](https://draw.io/diagramas/ciclo-de-ingesta.drawio).

Lo que no es obvio:

- **Dos commits, no uno.** La prensa se guarda antes de las cosechas pagadas. Del 21 al 24 de septiembre las 22 corridas murieron dentro de TikTok y se llevaron la prensa con ellas.
- **Cada paso pagado tiene su propio límite de tiempo y `continue-on-error`.** El trabajo entero tiene 75 minutos, por encima de la suma, para que siempre salte primero el límite del paso y se guarde lo demás.
- **Las cosechas llaman a Apify en paralelo** (seis a la vez) y aplican resultados en orden: la salida sigue siendo idéntica byte a byte.
- **El commit del robot no dispara otro flujo** (los commits con `GITHUB_TOKEN` no disparan `on: push`). Por eso la ingesta y el despliegue no se separan en dos flujos.

- **Determinismo.** Dos corridas sobre la misma entrada producen archivos idénticos; si no, cada corrida haría un commit de ruido. Cualquier serie nueva va ordenada y con su clave explícita.

Secretos y variables del repositorio

Nombre	Tipo	Para qué
<code>APIFY_TOKEN</code>	Secreto	Cosechas de redes
<code>YOUTUBE_API_KEY</code>	Secreto	Panel de conversación (apagado)
<code>VERCEL_TOKEN</code> , <code>VERCEL_ORG_ID</code> , <code>VERCEL_PROJECT_ID</code>	Secretos	Despliegue desde el runner
<code>APIFY_HABILITADO</code>	Variable	<code>true</code> enciende todas las cosechas pagadas
<code>YOUTUBE_HABILITADO</code>	Variable	Panel de conversación; apagado pendiente de opinión legal
<code>DESPLEGAR_TABLERO</code>	Variable	Despliegue desde el runner (hoy sin definir)

CI

`.github/workflows/ci.yml` corre en cada PR: pruebas de Python, `validar`, una corrida completa sin red a un directorio temporal, `validar` sobre esa salida, el sitio anterior, y en `web/`: `pnpm tipos`, diez scripts `probar-*.cjs`, `pnpm tokens` y `pnpm build`. Sin secretos. Debe estar en verde antes de pedir revisión.

4. Pruebas

```
python -m unittest discover -s tests -v
```

34 módulos, ~990 pruebas, **todas sin red** y sin cargar el modelo (se usa `AnalizadorFalso`). Leen los `config/*.json` reales, así que editar un catálogo puede romper una prueba legítimamente: revisa eso antes de culpar al código.

```
pnpm --dir web tipos
```

```
node web/scripts/probar-seguimiento.cjs
```

Hay once `probar-*.cjs` (análisis, aprobación, búsqueda, capítulos, consultas, garitas, informe, publicaciones, publicidad-meta, redes-en-vivo, seguimiento). CI corre diez; `probar-publicidad-meta.cjs` no.

5. Despliegue

Cómo está hoy

La integración Git de Vercel construye cada commit de `main`, incluidos los del robot. Cada ingesta produce un despliegue de producción en 24–48 s, que corre `sincronizar-datos.mjs` sobre el `data/` nuevo. Comprobarlo:

```
vercel ls pulso --scope areyes-1125
```

```
vercel inspect <url> --logs --scope areyes-1125
```

Consecuencia: el sitio publicado sale **sin texto de comentarios**, porque ese texto no está en git. Los paneles lo dicen: «El texto de los comentarios no está disponible en esta vista». No es un error del panel.

El despliegue desde el runner (apagado)

Es la única forma de publicar con comentarios. Para encenderlo:

1. Crear los secretos `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`.
2. Definir la variable `DESPLEGAR_TABLERO=true`.

Con eso el runner construye y despliega con el texto que acaba de escribir, y el robot añade `Despliegue: runner` a sus commits para que `web/vercel.json` (`ignoreCommand`) le diga a Vercel que no construya también esos commits. Definir solo la variable, sin los secretos, hace fallar el paso.

Reglas

- **Nunca hagas un commit vacío para redespregar:** dispara una ingesta completa, actores pagados incluidos.
- No escribas el token de salto de CI en el mensaje de un commit que sí quieres que corra, ni siquiera citado en el cuerpo: GitHub lee el mensaje entero.
- Variables del tablero: en Vercel → Settings → Environment Variables. La lista completa y comentada está en `web/.env.example`.

Migraciones de Neon

```
DATABASE_URL="postgresql://..." pnpm --dir web migrar
```

Aplica **todos** los `web/db/*.sql` en orden, cada vez; no hay bitácora de migraciones, así que cada archivo debe ser idempotente. Alternativa: pegar el SQL en el editor de Neon.

El servicio de tono (pulso- tono)

Proyecto de Vercel aparte, en `https://pulso- tono.vercel.app/api/tono`.

```
node servicio-tono/empaquetar.mjs
```

y después, **dentro de** `servicio-tono/`:

```
vercel deploy --prod --scope areyes-1125
```

Nunca desde la raíz del repositorio: `vercel deploy` ignora `.gitignore` y subiría `cache/` con texto crudo de comentarios. En el proyecto de Vercel: `TONO_EMPAQUETAR=1`, `VERCEL_SUPPORT_LARGE_FUNCTIONS=1` y `TONO_SECRETO`. En el tablero: `TONO_URL` y el mismo `TONO_SECRETO`. Salud: `GET /api/tono?salud=1` con el secreto en `X-Tono-Secreto`. Arranque en frío ~10 s, luego ~120 ms por texto.

En local: `python -m pulso tono --servir` y `TONO_URL=http://127.0.0.1:8765/` en `web/.env.local`.

6. Variables de entorno del tablero

Variable	Efecto
<code>AUTH_SECRET</code>	Firma de la sesión; también la clave del HMAC de búsquedas
<code>AUTH_MICROSOFT_ENTRA_ID_ID</code> , <code>_SECRET</code> , <code>_ISSUER</code>	Las tres o no hay entrada
<code>ACCESO_PRIMER_ADMIN</code>	Correo que siempre es admin
<code>AVISO_SOLICITUD_REMITENTE</code> , <code>AVISO_SOLICITUD_PARA</code>	Correo a admins cuando alguien pide acceso
<code>DATABASE_URL</code> (o <code>NEON_DB_DATABASE_URL</code>)	Neon
<code>ANALISIS_HABILITADO=true</code> + <code>ANTHROPIC_API_KEY</code>	Analizar, Detallar, Guion, resumen de comentarios
<code>SEGUIMIENTO_HABILITADO=true</code> + <code>APIFY_API_TOKEN</code> + base	Botón «Actualizar» de Seguimiento
<code>BUSQUEDA_REDES_HABILITADA=true</code> + Apify + base + <code>AUTH_SECRET</code>	Búsqueda pagada en vivo (sin botón en pantalla desde el 30 de septiembre)
<code>TONO_URL</code> , <code>TONO_SECRETO</code>	Tono de comentarios; sin ellos todo sale «sin tono»
<code>CRON_SECRET</code>	Protege <code>/api/seguimiento/purgar</code>
<code>NEXT_PUBLIC_DATOS_URL</code>	Ruta de los JSON (por omisión <code>/data</code>); se fija al construir

7. Diagnóstico

«El sitio está viejo»

No mires el sitio, mira el archivo:

```
git log origin/main -1 -- data/tiktok.json
```

y su campo `generado`. Si la prensa está al día y las redes no, casi siempre es `APIFY_HABILITADO` **apagado** o un paso pagado que murió por tiempo. Revisa la última corrida en Actions: las anotaciones amarillas lo dicen. `pulso validar` avisa cuando un panel lleva más de 12 h detrás de `estado.json`, y las tarjetas de Redes muestran la fecha en lugar de «hace 2 h» en ese caso.

En local, si el guion sale sin clips: `web/public/data` es de cuando arrancaste `pnpm dev`. `pnpm --dir web datos`.

Tabla de síntomas

Síntoma	Causa probable	Qué hacer
Redes sin comentarios en producción	El sitio lo construye Vercel desde git	Esperado; encender el despliegue desde el runner (\$5)
Todas las cuentas de redes «ok» pero vacías	Falta el token de Apify	<code>python -m pulso apify --verificar</code>
La corrida sale con código 2	Ninguna fuente de prensa respondió	Revisar red del runner; es un apagón, no una falla parcial
<code>validar</code> falla tras editar <code>config/</code>	Fila activa sin <code>verificado</code> , <code>zona</code> y <code>ambito</code> a la vez, <code>marca</code> duplicada	Leer el mensaje: dice la regla y el porqué
Cada corrida hace commit aunque no haya noticias	Algo no determinista (orden de un set, una hora)	Encontrarlo; <code>TestDeterminismo</code> debería atraparlo
<code>redirect_uri_mismatch</code> al entrar	URI de redirección mal registrada en Azure	Debe ser <code>.../api/auth/callback/microsoft-entra-id</code> , tipo Web
<code>AADSTS7000215</code>	Se copió el «Id. de secreto» en vez del valor	Generar otro secreto y copiar la columna Valor
Todo da 404 en Vercel	El proyecto no tiene el preset de Next.js	<code>docs/acceso.md</code> §7
API responde 401 / 403 / 503	Sin sesión / sin aprobar o desactivado / Neon caído	Revisar la cuenta en Accesos o el estado de Neon
Todo dice «sin tono»	Falta <code>TONO_URL</code> o <code>TONO_SECRETO</code> , o el servicio está frío	<code>GET /api/tono?salud=1</code>

Síntoma	Causa probable	Qué hacer
Guion o Analizar no aparecen	<code>ANALISIS_HABILITADO</code> o la llave faltan	Es el comportamiento apagado: sin explicación en pantalla
Las líneas de <code>data/</code> cambian solas en Windows	Finales de línea	<code>.gitattributes</code> fuerza LF; no lo quites